Práctica 1 — Identificación del Motor CC

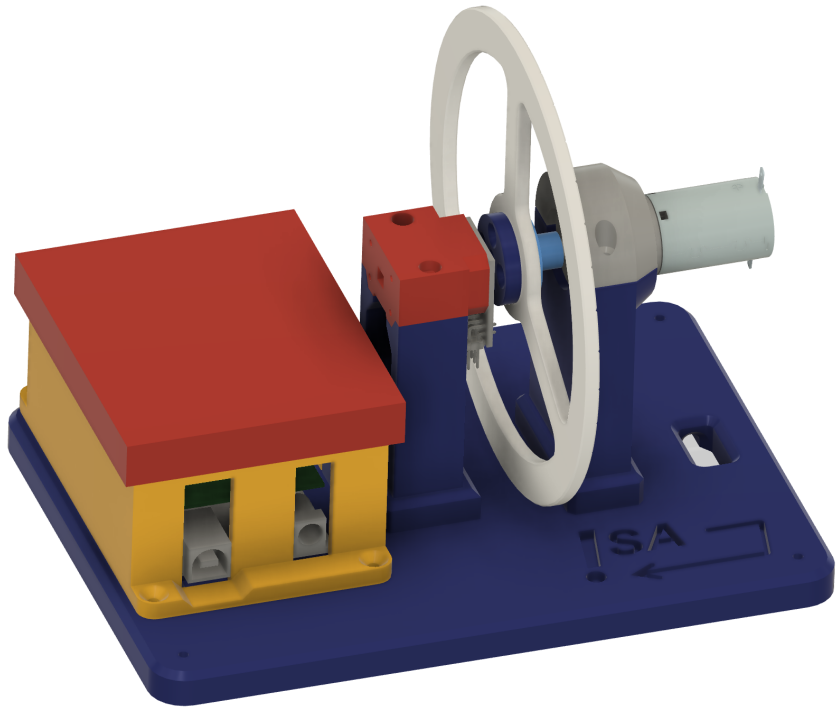
Dpto. de Ingeniería de Sistemas y Automática
Escuela Superior de Ingenieros — Universidad de Sevilla

Índice

1. Introducción al sistema
   - El equipo ISA
   - Modelo matemático
   - Interfaz Simulink
2. Laboratorio virtual con Simulink
   - Simulación vs. lab. virtual
   - Configuración
   - Controles de entrada
   - Indicadores de salida
3. Puesta en marcha
   - Pasos de puesta en marcha
   - Verificación
4. Experimentos
   - Ejercicio previo
   - Experimento 1 — Saturación
   - Experimento 2 — Característica estática
   - Experimento 3 — Ensayo en escalón

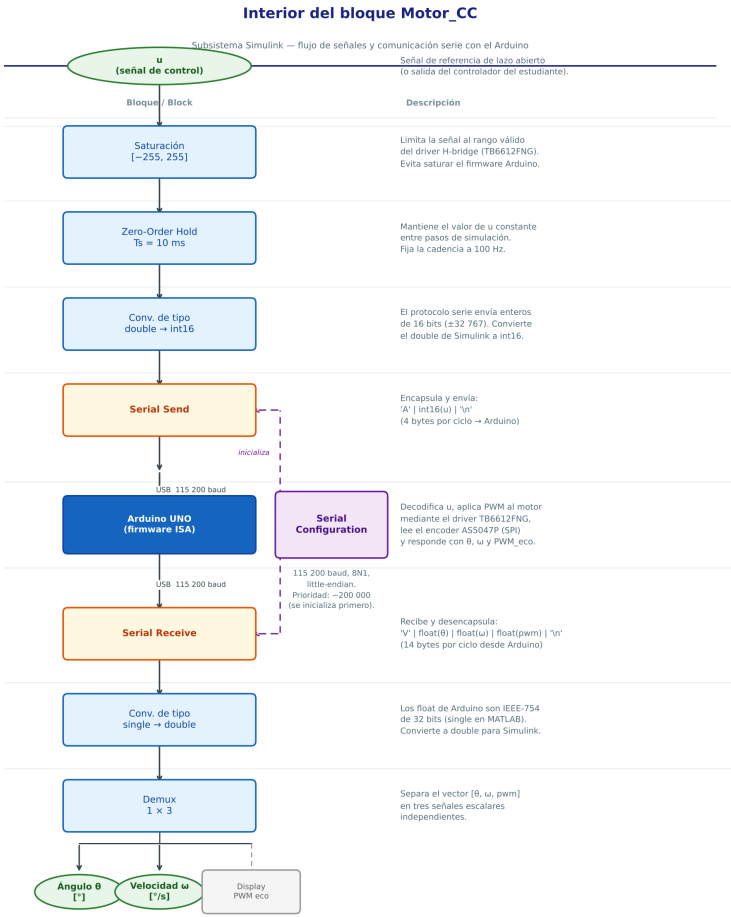
El equipo ISA

El equipo de prácticas ha sido diseñado íntegramente por el Dpto. ISA. Integra en una placa compacta (*shield*) sobre Arduino UNO todos los elementos para controlar un motor CC real desde MATLAB/Simulink.



Componentes del equipo

- **Motor CC:** Maxon A-max 22, 5 W, 12 V con reductora GP 22B.
- **Encoder:** AS5047P, 14 bits (16 384 pasos/vuelta), magnético.
- **Driver:** TB6612FNG (puente en H dual). $u \in [-255, 255]$.
- **Arduino UNO:** lee encoder (SPI), genera PWM, comunica con PC.
- **PC con MATLAB/Simulink:** ejecuta el controlador en tiempo real.



Modelo del motor CC

El conjunto motor+reductora se modela como un sistema de **primer orden**:

Función de transferencia

$$G(s) = \frac{\Omega(s)}{V_{eq}(s)} = \frac{K}{\tau s + 1}$$

Parámetros a identificar:

- K [°/s / V]: ganancia estática.
- τ [s]: constante de tiempo mecánica.

Señal de control:

$$V_{eq} = u \cdot \frac{V_{alim}}{255}$$

$u \in [-255,\ 255]$; V_{alim} depende del modo:

- **Batería:** $V_{alim} \approx 8{,}4\,\text{V}$ ($V_{max} = 9\,\text{V}$)
- **USB:** $V_{alim} = 5\,\text{V}$ (rango

El bloque Motor_CC

Toda la comunicación con el Arduino está encapsulada en el bloque `Motor_CC` del modelo `MotorCC_V3.slx`.

Señales del bloque

Dirección	Señal	Descripción
Simulink → Arduino	u	Control $[-255,\ 255]$
Arduino → Simulink	θ [°]	Posición angular acumulada
Arduino → Simulink	ω [°/s]	Velocidad angular

- Período de muestreo: $T_s = 10\,\text{ms}$ $(100\,\text{Hz})$.
- $u > 0 \Rightarrow$ un sentido de giro; $u < 0 \Rightarrow$ sentido contrario; $u = 0 \Rightarrow$ parado.

Simulación vs. laboratorio virtual

Simulación clásica

- Tiempo determinado (inicial–final).
- Entradas predefinidas.
- Análisis *a posteriori*.

Laboratorio real/virtual

- Stop Time = `Inf`.
- Modificación manual de u en tiempo real.
- Visualización inmediata.

En esta práctica

El modelo Simulink se comunica con el Arduino a 100 Hz. Podemos cambiar u mientras el motor está en marcha y ver la respuesta al instante.

Configuración de la simulación

Para trabajar en modo *tiempo real interactivo*:

1. Establezca **Stop Time** = `Inf`.
2. `Run` → flecha desplegable → *Simulation Pacing*.
3. Active *Enable pacing to slow down simulation*.
4. Escoja **1 s simulado / 1 s real**.

Nota

Con el equipo real, la sincronía la fija el Arduino (100 Hz); el *pacing* evita que Simulink adelante los cálculos.

Controles de entrada — Dashboard

Los bloques `Simulink / Dashboard` permiten modificar parámetros de cualquier bloque *sin detener la simulación*:

Controles disponibles:

- **Knob** — dial giratorio.
- **Edit** — campo de texto numérico.
- **Slider** — deslizador.

Cómo conectar:

1. Doble clic sobre el control.
2. Seleccionar el bloque destino.
3. Elegir el parámetro de la lista.

Uso en esta práctica:

- **Knob** o **Edit** conectado al bloque `Constant` para fijar u.
- `Manual Switch` para conmutar entre constante y senoidal.

Scope y Dashboard Scope

Scope (Sinks)

- `View` → `Configuration`:
  - *Time span*: no usar `Auto`.
  - *Update mode*: Wrap o Scroll.
  - *Y-Axis*: fijar mín. y máx.

Dashboard Scope

- Se incrusta dentro del diagrama.
- Doble clic → seleccionar señal.
- Ideal para supervisar θ y ω a la vez.

Data Inspector

`Simulation` → `Data Inspector`: registra todas las señales para análisis posterior. Dar nombre a cada señal y activar el icono de registro (*log*).

Puesta en marcha del equipo

1. **Conecte el Arduino** al PC por USB. Compruebe el LED de 12 V.
2. **Abra MATLAB** y navegue a `software/matlab/`.
3. **Abra el modelo:** doble clic en `MotorCC_V3.slx`.
4. **Seleccione el puerto serie:** doble clic en el bloque `Motor_CC` → desplegable *Puerto serie* → seleccione el puerto del Arduino → **OK**.
5. **Pulse Run** (▷). La simulación arranca en tiempo real.
6. **Verifique:** el Scope debe mostrar θ y ω actualizándose.

Verificación del equipo

Con el modelo en marcha:

1. Fije $u = 50$ en el bloque `Constant`. El motor debe girar; θ sube (rampa), $\omega \approx$ cte.
2. Cambie a $u = -50$: el motor gira en sentido contrario.
3. Fije $u = 0$: el motor se detiene.
4. Configure Stop Time = `Inf` y pacing 1:1 para los experimentos.

Si el motor no responde

- Verifique el puerto serie en el bloque `Motor_CC`.
- Compruebe que la fuente de 12 V está encendida.
- Stop $\rightarrow$ Run (reinicio del modelo).

Ejercicio 1 — Preguntas y script de inicialización

Preguntas teóricas

1. ¿Cuál es la **entrada** del sistema motor CC?
2. ¿Cuáles son las **salidas** medidas?
3. ¿Qué ocurre si $|u| > 255$?

Script `init_motor.m`

```
% init_motor.m --- Parametros del equipo ISA Motor CC
u_max   = 255;    % Valor maximo de la senal de control
u_min   = -255;   % Valor minimo de la senal de control
Ts      = 0.01;   % Periodo de muestreo [s]
V_alim = 8.4;     % 8.4 V bateria (max 9V); 5V si USB
```

Experimento 1 — Efecto de la saturación en u

Procedimiento

Cambie u progresivamente a través de los valores:

$$u \in \{-300,\ -255,\ -128,\ 0,\ 128,\ 255,\ 300\}$$

Espere unos segundos en cada valor para que ω se estabilice.

Grafique u y ω en un Scope con dos paneles y haga captura de pantalla.

Efecto de saturación en u

¿Cambia ω al pasar de $u = 255$ a $u = 300$? ¿Y de $u = -255$ a $u = -300$?

Experimento 1 — Gráfica y tabla

[Pegue aquí la captura del Scope]

u	ω_∞ [°/s]
-255	
-128	
0	
128	
255	

Experimento 2 — Característica estática

Procedimiento

Mida ω_∞ en régimen permanente para:

$$u \in \{-255,\ -200,\ -128,\ -64,\ 0,\ 64,\ 128,\ 200,\ 255\}$$

Script MATLAB

```
u_vec = [-255,-200,-128,-64, 0, 64,128,200,255];
w_vec = [   ?,   ?,   ?,  ?, ?,  ?,  ?,  ?,  ?]; % rellenar
plot(u_vec, w_vec, 'o-', 'LineWidth', 1.5)
xlabel('Senal de control u [-]')
ylabel('\omega_\infty [deg/s]')
grid on;  title('Caracteristica estatica del motor CC')
```

Experimento 2 — Gráfica y preguntas

[Pegue aquí la gráfica de la característica estática]

Preguntas

1. ¿Es la relación $u \to \omega_\infty$ lineal? ¿Existe zona muerta?
2. Estime la ganancia estática en la zona lineal: $K \approx \Delta\omega_\infty / \Delta V_{eq}$ [°/s / V].

Experimento 3 — Ensayo en escalón

Procedimiento

1. Parta de $u = 0$ con el motor parado.
2. Aplique un escalón a $u = 128$ ($\approx 6\,\mathrm{V}$).
3. Espere hasta que ω alcance el régimen permanente ($\approx 5\tau$).
4. Repita con $u = 200$ y $u = -128$.

Grafique u (panel superior) y ω (panel inferior). Haga captura de pantalla de cada ensayo.

Experimento 3 — Identificación de K y τ

Ganancia K

$$K = \frac{\omega_\infty}{V_{eq}} = \frac{\omega_\infty}{u \cdot V_{alim}/255}$$

Constante de tiempo τ

Tiempo en que ω alcanza el **63,2 %** de su valor final, medido desde el escalón.

Magnitud	Valor
V_{alim} [V]	
u [-]	
V_{eq} [V]	
ω_∞ [°/s]	
K [°/s / V]	
τ [s]	

Experimento 3 — Gráfica y validación

[Pegue aquí la captura del Scope — respuesta al escalón]

Validación con MATLAB

```
K = ?;  tau = ?;            % valores identificados
s = tf('s');
G = K / (tau*s + 1);
u0 = 128 * V_alim/255;      % tension equivalente
step(u0*G)
hold on
% superponga los datos experimentales del Data Inspector
```

Experimento 4 — Barrido de frecuencia (opcional)

Procedimiento

1. Conecte u a una señal senoidal: amplitud $A = 100$, bias $= 0$.
2. Comience con $f = 0{,}1\,\text{Hz}$ y aumente: 0,2 — 0,5 — 1 — 2 — 5 Hz.
3. Observe en el Scope la amplitud de ω en cada frecuencia.

Pregunta

¿A partir de qué frecuencia cae la amplitud de ω? Compare con la frecuencia de corte teórica $f_c = 1/(2\pi\tau)$.

Resumen de la práctica

Resultados esperados

1. **Característica estática:** gráfica u vs. ω_∞.
2. **Respuesta al escalón:** capturas del Scope (mínimo dos escalones).
3. **Parámetros identificados:** K [°/s / V] y τ [s].
4. **Función de transferencia:** $G(s) = K/(\tau s + 1)$ validada.

Próximas prácticas

- **Práctica 2:** diseño de un regulador PID.
- **Práctica 3:** control por realimentación de estados.